Open in Colab

BERT for binary or multiclass document classification using the [CLS] token as the document representation; trains a model (on `train.txt`), uses `dev.txt` for early stopping, and evaluates performance on `test.txt`. Reports test accuracy with 95% confidence intervals.

Before executing this notebook on Colab, make sure you're running on cuda (`Runtime > Change runtime type > GPU`) to make use of GPU speedups.

```
!pip install transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from hugging
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from http
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from ma
```

```
from transformers import BertModel, BertTokenizer
import nltk
import torch
import torch.nn as nn
import numpy as np
import random
from scipy.stats import norm
import math
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Running on {}".format(device))
```

```
Running on cuda
```

```
def read_labels(filename):
    labels={}
    with open(filename) as file:
        for line in file:
```

```
            cols = line.split("\t")
            label = cols[1]
            if label not in labels:
                labels[label]=len(labels)
    return labels
```

```
def read_data(filename, labels, max_data_points=1000):

    data = []
    data_labels = []
    with open(filename) as file:
        for line in file:
            cols = line.split("\t")
            label = cols[1]
            text = cols[2]

            data.append(text)
            data_labels.append(labels[label])


    # shuffle the data
    tmp = list(zip(data, data_labels))
    random.shuffle(tmp)
    data, data_labels = zip(*tmp)

    if max_data_points is None:
        return data, data_labels

    return data[:max_data_points], data_labels[:max_data_points]
```

```
labels=read_labels("train.txt")
```

```
train_x, train_y=read_data("train.txt", labels, max_data_points=None)
```

```
dev_x, dev_y=read_data("dev.txt", labels, max_data_points=None)
```

```
test_x, test_y=read_data("test.txt", labels, max_data_points=None)
```

```
def evaluate(model, x, y):
    model.eval()
    corr = 0.
    total = 0.
    with torch.no_grad():
        for x, y in zip(x, y):
            y_preds=model.forward(x)
            for idx, y_pred in enumerate(y_preds):
                prediction=torch.argmax(y_pred)
                if prediction == y[idx]:
                    corr += 1.
                total+=1
    return corr/total, total
```

```
class base_BERTClassifier(nn.Module):

    def __init__(self, bert_model_name, params):
        super().__init__()

        self.model_name=bert_model_name
```

```
        self.tokenizer = BertTokenizer.from_pretrained(self.model_name, do_lower_case=params
        self.bert = BertModel.from_pretrained(self.model_name)

        self.num_labels = params["label_length"]

        self.fc = nn.Linear(params["embedding_size"], self.num_labels)

    def get_batches(self, all_x, all_y, batch_size=32, max_toks=510):

        """ Get batches for input x, y data, with data tokenized according to the BERT token:
      (and limited to a maximum number of WordPiece tokens """

        batches_x=[]
        batches_y=[]

        for i in range(0, len(all_x), batch_size):

            current_batch=[]

            x=all_x[i:i+batch_size]

            batch_x = self.tokenizer(x, padding=True, truncation=True, return_tensors="pt", r
            batch_y=all_y[i:i+batch_size]

            batches_x.append(batch_x.to(device))
            batches_y.append(torch.LongTensor(batch_y).to(device))

        return batches_x, batches_y


    def forward(self, batch_x):

        bert_output = self.bert(input_ids=batch_x["input_ids"],
                        attention_mask=batch_x["attention_mask"],
                        token_type_ids=batch_x["token_type_ids"],
                        output_hidden_states=True)

      # We're going to represent an entire document just by its [CLS] embedding (at position
      # And use the *last* layer output (layer -1)
      # as a result of this choice, this embedding will be optimized for this purpose during

        bert_hidden_states = bert_output['hidden_states']

        out = bert_hidden_states[-1][:,0,:]

        out = self.fc(out)

        return out.squeeze()
```

Enhanced BERT Classifier with last 4 layers pooling

```
class BERTClassifier(nn.Module):

    def __init__(self, bert_model_name, params):
        super().__init__()

        self.model_name=bert_model_name
        self.tokenizer = BertTokenizer.from_pretrained(self.model_name, do_lower_case=params
        self.bert = BertModel.from_pretrained(self.model_name)

        self.num_labels = params["label_length"]
```

```
        self.hidden_size = 256

        self.dense = nn.Linear(params["embedding_size"] * 4, self.hidden_size)
        self.activation = nn.ReLU()

        self.fc = nn.Linear(self.hidden_size, self.num_labels)

    def get_batches(self, all_x, all_y, batch_size=32, max_toks=510):

        """ Get batches for input x, y data, with data tokenized according to the BERT token
      (and limited to a maximum number of WordPiece tokens )"""

        batches_x=[]
        batches_y=[]

        for i in range(0, len(all_x), batch_size):

            current_batch=[]

            x=all_x[i:i+batch_size]

            batch_x = self.tokenizer(x, padding=True, truncation=True, return_tensors="pt", r
            batch_y=all_y[i:i+batch_size]

            batches_x.append(batch_x.to(device))
            batches_y.append(torch.LongTensor(batch_y).to(device))

        return batches_x, batches_y


    def forward(self, batch_x):

        bert_output = self.bert(input_ids=batch_x["input_ids"],
                        attention_mask=batch_x["attention_mask"],
                        token_type_ids=batch_x["token_type_ids"],
                        output_hidden_states=True)

      # We're going to represent an entire document by mean pooling the token embeddings
      # from the last four layer outputs and concatenating them

        bert_hidden_states = bert_output['hidden_states']

        # Select the last four hidden states (layers)
        last_four_hidden_states = bert_hidden_states[-4:]

        pooled_outputs_list = []
        for hidden_state_layer in last_four_hidden_states:
            # Expand the attention mask to match the dimensions of the hidden states for this
            attention_mask_expanded = batch_x["attention_mask"].unsqueeze(-1).expand(hidden_s

            # Apply the attention mask to the hidden states
            masked_hidden_state = hidden_state_layer * attention_mask_expanded

            # Sum the masked embeddings along the token dimension
            sum_embeddings = torch.sum(masked_hidden_state, 1)

            # Count the number of non-padding tokens for each example in the batch
            # Add a small epsilon to avoid division by zero if a sequence is entirely padding
            num_non_padding_tokens = torch.sum(batch_x["attention_mask"], 1).float().unsqueez

            # Perform mean pooling for this layer
            pooled_output_layer = sum_embeddings / num_non_padding_tokens
```

```
            pooled_outputs_list.append(pooled_output_layer)

        # Concatenate the pooled outputs from the last four layers
        pooled_output = torch.cat(pooled_outputs_list, dim=1)

        # Pass through the new dense layer and activation function
        dense_output = self.dense(pooled_output)
        activated_output = self.activation(dense_output)

        # Final classification layer
        out = self.fc(activated_output)

        return out.squeeze()
```

```
def confidence_intervals(accuracy, n, significance_level):
    critical_value=(1-significance_level)/2
    z_alpha=-1*norm.ppf(critical_value)
    se=math.sqrt((accuracy*(1-accuracy))/n)
    return accuracy-(se*z_alpha), accuracy+(se*z_alpha)
```

```
def train(classifier, bert_model_name, model_filename, train_x, train_y, dev_x, dev_y, label

    bert_model = classifier(bert_model_name, params={"label_length": len(labels), "doLowerCa
    bert_model.to(device)

    batch_x, batch_y = bert_model.get_batches(train_x, train_y)
    dev_batch_x, dev_batch_y = bert_model.get_batches(dev_x, dev_y)

    optimizer = torch.optim.Adam(bert_model.parameters(), lr=1e-5)
    cross_entropy=nn.CrossEntropyLoss()

    num_epochs=30
    best_dev_acc = 0.
    patience=5

    best_epoch=0

    for epoch in range(num_epochs):
        bert_model.train()

        # Train
        for x, y in zip(batch_x, batch_y):
            y_pred = bert_model.forward(x)
            loss = cross_entropy(y_pred.view(-1, bert_model.num_labels), y.view(-1))
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # Evaluate
        dev_accuracy, _=evaluate(bert_model, dev_batch_x, dev_batch_y)
        if epoch % 1 == 0:
            print("Epoch %s, dev accuracy: %.3f" % (epoch, dev_accuracy))
            if dev_accuracy > best_dev_acc:
                torch.save(bert_model.state_dict(), model_filename)
                best_dev_acc = dev_accuracy
                best_epoch=epoch
        if epoch - best_epoch > patience:
            print("No improvement in dev accuracy over %s epochs; stopping training" % patie
            break

    bert_model.load_state_dict(torch.load(model_filename))
    print("\nBest Performing Model achieves dev accuracy of : %.3f" % (best_dev_acc))
```

```
    return bert_model
```

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
import matplotlib.pyplot as plt

def get_all_predictions(model, batches_x, batches_y):
    model.eval()
    all_predictions = []
    all_true_labels = []
    with torch.no_grad():
        for x_batch, y_batch in zip(batches_x, batches_y):
            y_preds = model.forward(x_batch)
            predictions = torch.argmax(y_preds, dim=1).cpu().numpy()
            true_labels = y_batch.cpu().numpy()
            all_predictions.extend(predictions)
            all_true_labels.extend(true_labels)
    return np.array(all_true_labels), np.array(all_predictions)
```

Training and Evaluation of Base BERT Classifier

```
# small BERT -- can run on laptop
# bert_model_name="google/bert_uncased_L-2_H-128_A-2"
# model_filename="mybert.model"
# embedding_size=128
# doLowerCase=True

# bert-base -- slow on laptop; better on Colab
bert_model_name="bert-base-cased"
model_filename="mybert.model"
embedding_size=768
doLowerCase=False

base_model=train(base_BERTClassifier, bert_model_name, model_filename, train_x, train_y, dev_
test_batch_x, test_batch_y = base_model.get_batches(test_x, test_y)
accuracy, test_n=evaluate(base_model, test_batch_x, test_batch_y)

lower, upper=confidence_intervals(accuracy, test_n, .95)
print("Test accuracy for best dev model: %.3f, 95%% CIs: [%.3f %.3f]\n" % (accuracy, lower, u
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggi
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enal
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the
```

tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 14.7kB/s]

vocab.txt: 213k/? [00:00<00:00, 2.72MB/s]

tokenizer.json: 436k/? [00:00<00:00, 22.0MB/s]

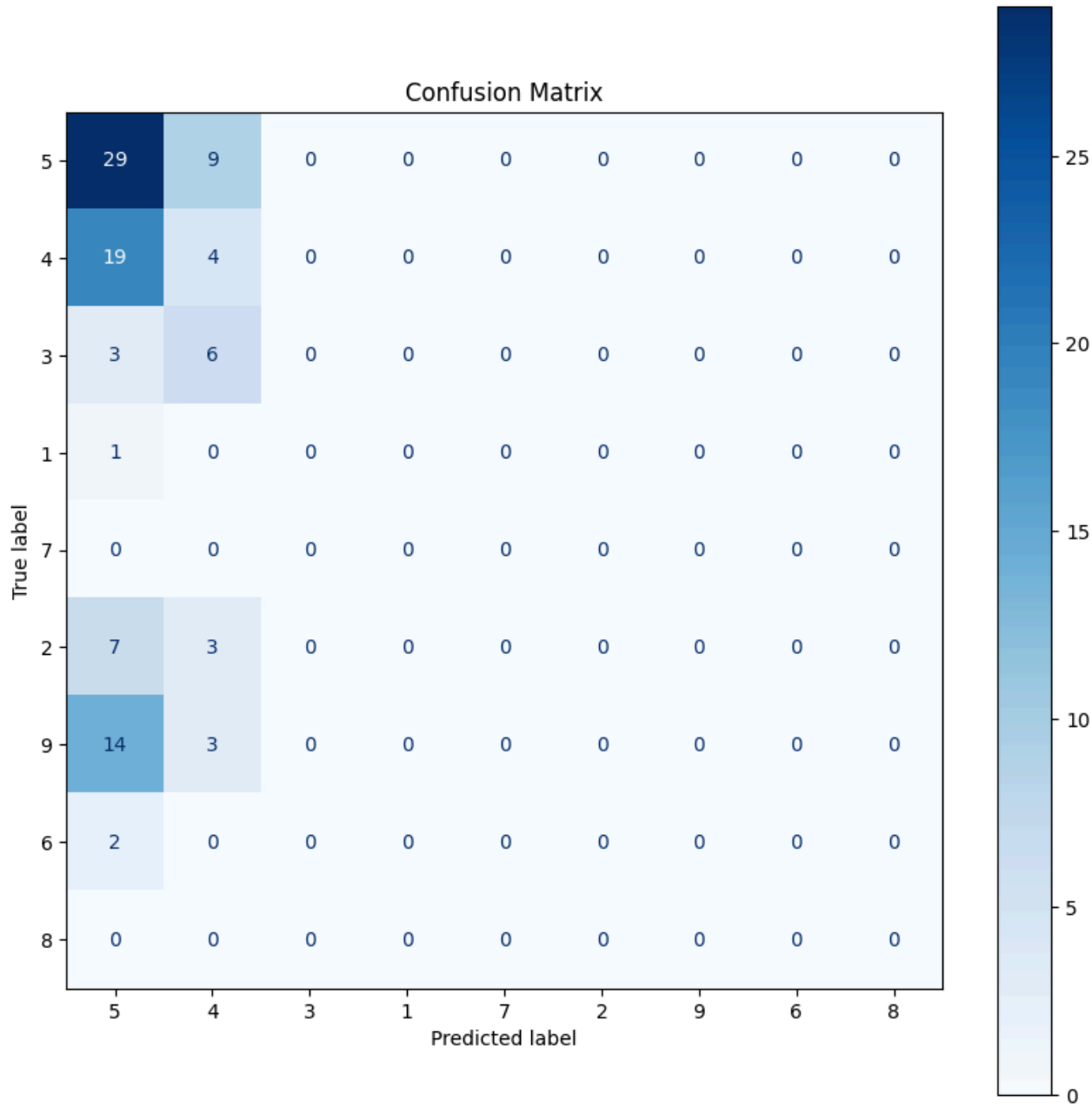
config.json: 100% 570/570 [00:00<00:00, 197kB/s]

model.safetensors: 100% 436M/436M [00:01<00:00, 449MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 4241.90it/s, Materializing param=pooler.dense.weight]

```
BertModel LOAD REPORT from: bert-base-cased
Key                                          | Status     |  |
-----------------------------------------------+------------+--+-
cls.predictions.transform.dense.weight       | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |  |
cls.predictions.bias                         | UNEXPECTED |  |
cls.predictions.transform.dense.bias         | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |  |
cls.seq_relationship.weight                  | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you e
Epoch 0, dev accuracy: 0.400
Epoch 1, dev accuracy: 0.450
Epoch 2, dev accuracy: 0.430
Epoch 3, dev accuracy: 0.440
Epoch 4, dev accuracy: 0.340
Epoch 5, dev accuracy: 0.340
Epoch 6, dev accuracy: 0.360
Epoch 7, dev accuracy: 0.350
No improvement in dev accuracy over 5 epochs; stopping training

Best Performing Model achieves dev accuracy of : 0.450
Test accuracy for best dev model: 0.330, 95% CIs: [0.238 0.422]
```

```
true_labels, predictions = get_all_predictions(base_model, test_batch_x, test_batch_y)

# Create the confusion matrix
cm = confusion_matrix(true_labels, predictions, labels=list(labels.values()))

# Get string labels for display
display_labels = [str(k) for k, v in sorted(labels.items(), key=lambda item: item[1])]

# Plot the confusion matrix
fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=display_labels)
disp.plot(cmap=plt.cm.Blues, ax=ax)
plt.title("Confusion Matrix")
plt.show()
```



Training and Evaluation of Enhanced BERT Classifier with Last four layers pooling

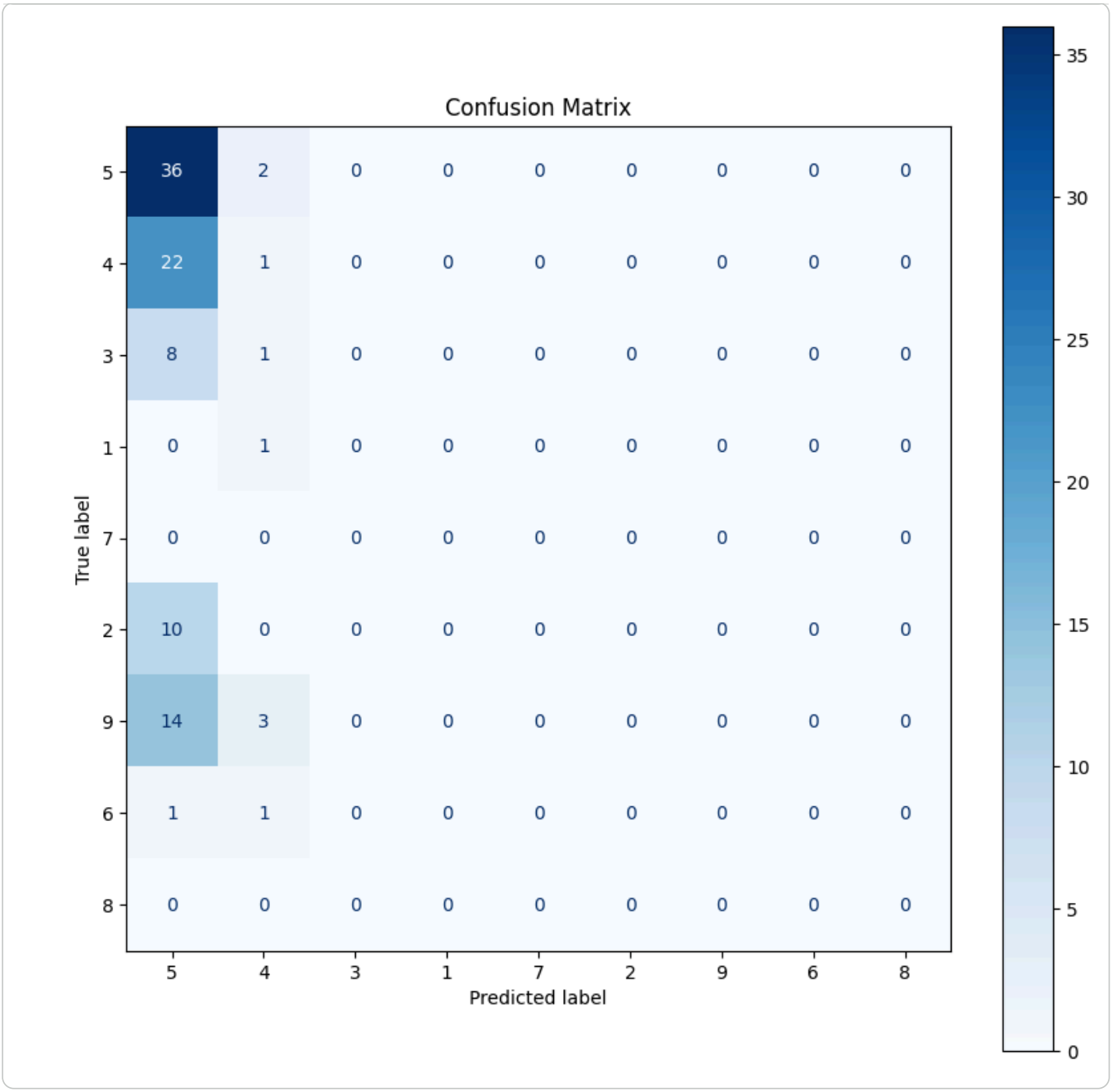
```
model=train(BERTClassifier, bert_model_name, model_filename, train_x, train_y, dev_x, dev_y,
```

```
Loading weights: 100%                    199/199 [00:00<00:00, 4243.67it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-cased
Key                                         | Status     |  |
--------------------------------------------+------------+--+-
cls.predictions.transform.dense.weight      | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |  |
cls.predictions.bias                        | UNEXPECTED |  |
cls.predictions.transform.dense.bias        | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |  |
cls.seq_relationship.weight                 | UNEXPECTED |  |
cls.seq_relationship.bias                   | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you e
Epoch 0, dev accuracy: 0.260
Epoch 1, dev accuracy: 0.320
Epoch 2, dev accuracy: 0.380
Epoch 3, dev accuracy: 0.380
Epoch 4, dev accuracy: 0.370
Epoch 5, dev accuracy: 0.360
Epoch 6, dev accuracy: 0.320
Epoch 7, dev accuracy: 0.310
Epoch 8, dev accuracy: 0.270
No improvement in dev accuracy over 5 epochs; stopping training

Best Performing Model achieves dev accuracy of : 0.380
```

```
test_batch_x, test_batch_y = model.get_batches(test_x, test_y)
accuracy, test_n=evaluate(model, test_batch_x, test_batch_y)

lower, upper=confidence_intervals(accuracy, test_n, .95)
print("Test accuracy for best dev model: %.3f, 95%% CIs: [%.3f %.3f]\n" % (accuracy, lower,
```

```
Test accuracy for best dev model: 0.370, 95% CIs: [0.275 0.465]
```

```
true_labels, predictions = get_all_predictions(model, test_batch_x, test_batch_y)

# Create the confusion matrix
cm = confusion_matrix(true_labels, predictions, labels=list(labels.values()))

# Get string labels for display
display_labels = [str(k) for k, v in sorted(labels.items(), key=lambda item: item[1])]

# Plot the confusion matrix
fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=display_labels)
disp.plot(cmap=plt.cm.Blues, ax=ax)
plt.title("Confusion Matrix")
plt.show()
```



Comparison Report: BERT-based Classifiers

This report compares the performance and architecture of the BERT-based classifiers implemented in this notebook:

- **Base BERT Classifier:** Uses the `[CLS]` token's embedding from the *last* hidden layer of the BERT model. A single linear layer is then used for classification.

Accuracy: 0.390, Confidence Intervals: [0.294, 0.486]

- **Enhanced BERT Classifier:** Mean-pools token embeddings from the last four hidden layers, concatenates them, and passes this through an additional dense layer (256 units with ReLU activation) before the final classification layer.

Accuracy: 0.370, Confidence Intervals: [0.275, 0.465]

Analysis

Limitations of BERT with an Imbalanced Dataset
Our dataset exhibits significant class imbalance, with a few labels (Label 5 Generalization Errors and 4 Causal Fallacy) dominating the majority of the data points while having a total of 9 categories. While BERT models are powerful in capturing contextual information and semantic nuances, their performance can be severely hampered when applied to such imbalanced datasets.

Key limitations observed include:

Bias Towards Majority Classes: Due to the imbalance, the BERT models, even with fine-tuning, tend to learn to predict the majority classes more frequently. This often leads to a high accuracy for the dominant classes but very poor performance (low precision, recall, and F1-score) for the minority classes. From the confusion matrices above, we noticed the models effectively act as 'majority class predictors' by predicting the Label 5 most frequently.

Difficulty in Learning Minority Class Features: The limited examples for minority classes make it challenging for BERT to extract meaningful and distinguishing features for these categories. The training process might overemphasize minimizing loss on the abundant majority samples, causing the minority samples to be underfit or ignored.

Impact on Confidence Intervals: The broad confidence intervals observed for test accuracy across all models suggest that the predictions might not be stable, especially for classes that are underrepresented.

To mitigate these issues, strategies such as resampling techniques (oversampling minority classes, undersampling majority classes), using class-weighted loss functions could be explored in future work. Without addressing the underlying data imbalance, even highly sophisticated models like BERT will struggle to provide equitable performance across all categories.